# Tioskop – Lihat & Booking Film 🎟️

*A Functional Programming Approach with Rust*

**Authors:**
Kelompok 3 - Pemrograman Fungsional A
Aditya Ridho Nugroho | Alief Rachmattul Islam | Arya Zaky Pradipta | Muhamad Faisal | Muhammad Fatwa Al-Choiri

## Abstract

Tioskop adalah aplikasi jadwal dan booking bioskop yang dibangun menggunakan Rust sebagai Backend dan Vue.js sebagai Frontend dengan pendekatan *functional programming*. Backend dikembangkan menggunakan framework **Axum** dan runtime asynchronous **Tokio**, memungkinkan sistem menangani request secara *concurrent*, *async*.

## Introduction

Aplikasi ini dirancang untuk menyelesaikan permasalahan utama pada sistem jadwal bioskop umumnya yatiu:

- Pembatasan informasi jadwal yang tidak update dan lambat.
- Dibutuhkan sistem modern dengan arsitektur aman, efisien, dan scalable.

### Mengapa Rust?

| Alasan | Penjelasan |
|---|---|
| Efisiensi memory | Mengurangi crash pada booking concurrency. |
| High concurrency | Cocok untuk sistem jadwal & booking yang banyak request. |
| Functional friendly | Mendukung paradigma pemrograman fungsional. |

### Tujuan Utama

- Memberikan sistem manajemen bioskop yang cepat, scalable, dan aman.
- Menyediakan API lihat dan booking film yang cepat dan tepat.
- Mengaplikasikan paradigma **Functional Programming** dalam implementasi pengembangan sistem.

## Background & Concepts

### Technology Stack

| Komponen | Teknologi |
|---|---|
| Backend | **Rust + Axum** |

| Komponen | Teknologi |
|---|---|
| FrontEnd | **VueJS + TailwindCSS** |
| Runtime Async | **Tokio** |
| Database | MySQL |
| JSON Serialization | Serde |
| Time & Date | Chrono |
| Numeric & Decimal | rust_decimal |
| CORS Config | tower-http |

### Konsep Functional Programming Dalam Sistem

| Konsep FP | Implementasi Dalam Proyek |
|---|---|
| Pure Function | Perhitungan harga, validasi seat, transformasi data API |
| Pattern Matching | Handling error + branch booking logic |

Dengan ini aplikasi bisa menangani ratusan request booking serentak tanpa konflik seat.

---

## Source Code Overview

### Struktur Folder Project

```
tioskop/
├── backend/                        # Backend
│   ├── src/
│   │   ├── main.rs                 # Entry point aplikasi
│   │   │
│   │   ├── config/
│   │   │   └── mod.rs              # Database connection pool
│   │   │
│   │   ├── models/                 # Data models
│   │   │   ├── mod.rs
│   │   │   ├── movie.rs
│   │   │   ├── showtime.rs
│   │   │   ├── studio.rs
│   │   │   ├── seat.rs
│   │   │   ├── booking.rs
│   │   │   └── response.rs
│   │   │
│   │   ├── handlers/               # Business logic layer
│   │   │   ├── mod.rs
│   │   │   ├── movie_handler.rs
│   │   │   ├── showtime_handler.rs
│   │   │   ├── studio_handler.rs
│   │   │   ├── seat_handler.rs
```

```
│   │   │   └── booking_handler.rs
│   │   │
│   │   └── routes/                  # Routing layer
│   │       ├── mod.rs
│   │       ├── movie_routes.rs
│   │       ├── showtime_routes.rs
│   │       ├── studio_routes.rs
│   │       ├── seat_routes.rs
│   │       └── booking_routes.rs
│   │
│   ├── target/
│   ├── Cargo.toml                   # Rust dependencies
│   ├── Cargo.lock
│   ├── .env                         # Environment variables
│   └── tioskop_db.sql               # Database schema & seed data
│
├── frontend/                        # Frontend
│   ├── src/
│   │   ├── main.js
│   │   ├── App.vue
│   │   ├── style.css
│   │   │
│   │   ├── components/              # Vue components
│   │   │   ├── Navbar.vue
│   │   │   ├── HeroSection.vue
│   │   │   ├── SearchSection.vue
│   │   │   └── NowShowingSection.vue
│   │   │
│   │   └── assets/                  # Static assets (images, icons)
│   │
│   ├── public/                      # Public static files
│   │   └── vite.svg
│   │
│   ├── node_modules/                # npm dependencies
│   ├── package.json
│   ├── package-lock.json
│   ├── vite.config.js
│   ├── index.html
│   ├── .gitignore
│
├── asset/                           # Project assets (screenshots, docs)
├── APIDoc.md                        # API documentation
└── README.md                        # Project documentation
```

---

## File Utama Backend

**src/main.rs**

Main point aplikasi yang menjalankan:

- Tokio async runtime menggunakan `#[tokio::main]`

- Database connection
- CORS middleware configuration untuk devlepoment
- Router dari semua module
- Jalankan Server di `127.0.0.1:3000`

**SC:**

```rust
mod config;
mod models;
mod handlers;
mod routes;

use axum::Router;
use dotenvy::dotenv;
use std::net::SocketAddr;
use tower_http::cors::{Any, CorsLayer};
use routes::{movie_routes::movie_routes, showtime_routes::showtime_routes,
studio_routes::studio_routes, seat_routes::seat_routes,
booking_routes::booking_routes};

#[tokio::main]
async fn main() {
    dotenv().ok();

    // Setup database
    let pool = config::create_pool().await;
    println!("Connected to database");

    // Setup CORS
    let cors = CorsLayer::new()
        .allow_origin(Any)
        .allow_methods(Any)
        .allow_headers(Any);

    // Build dengan routes
    let app = Router::new()
        .merge(movie_routes())
        .merge(showtime_routes())
        .merge(studio_routes())
        .merge(seat_routes())
        .merge(booking_routes())
        .layer(cors)
        .with_state(pool);

    // Run server
    let addr = SocketAddr::from(([127, 0, 0, 1], 3000));
    println!(" Server on http://{}", addr);

    let listener = tokio::net::TcpListener::bind(addr).await.unwrap();
    axum::serve(listener, app).await.unwrap();
}
```

**src/config/mod.rs**

Konfigurasi database connection pool menggunakan SQLx:

- Database URL dari environment variable
- Max connections: 10 concurrent connections
- MySQL connection pooling

**SC:**

```
001  use sqlx::{mysql::MySqlPoolOptions, MySqlPool};
002  use std::env;
003
004  pub async fn create_pool() -> MySqlPool {
005      let database_url = env::var("DATABASE_URL").expect("DATABASE_URL must be
set");
006
007      MySqlPoolOptions::new()
008          .max_connections(10)
009          .connect(&database_url)
010          .await
011          .expect("Failed to connect to database")
012  }
```

---

## Models Layer

**src/models/response.rs**

Membuat response wrapper untuk semua response API endpoints dengan kondisi succes dan error:

```
001  use serde::Serialize;
002
003  #[derive(Serialize)]
004  pub struct ApiResponse<T> {
005      pub success: bool,
006      pub message: String,
007      pub data: Option<T>,
008  }
009
010  #[derive(Serialize)]
011  pub struct DeleteResponse {
012      pub id: i64,
013      pub deleted: bool,
014  }
015
016  impl<T> ApiResponse<T> {
017      pub fn success(message: &str, data: T) -> Self {
018          ApiResponse {
```

```
            success: true,
            message: message.to_string(),
            data: Some(data),
        }
    }

    pub fn error(message: &str) -> Self {
        ApiResponse {
            success: false,
            message: message.to_string(),
            data: None,
        }
    }
}
```

**src/models/movie.rs**

Membuat model movie:

```
use serde::{Deserialize, Serialize};
use sqlx::FromRow;

#[derive(Serialize, FromRow, Clone)]
pub struct Movie {
    pub id: i64,
    pub title: String,
    pub genre: Option<String>,
    pub rating: Option<String>,
    pub duration: Option<i32>,
    pub description: Option<String>,
    pub poster_url: Option<String>,
    pub release_date: Option<chrono::NaiveDate>,
}

#[derive(Deserialize)]
pub struct CreateMovieRequest {
    pub title: String,
    pub genre: Option<String>,
    pub rating: Option<String>,
    pub duration: Option<i32>,
    pub description: Option<String>,
    pub poster_url: Option<String>,
    pub release_date: Option<chrono::NaiveDate>,
}

#[derive(Deserialize)]
pub struct UpdateMovieRequest {
    pub title: Option<String>,
    pub genre: Option<String>,
    pub rating: Option<String>,
    pub duration: Option<i32>,
```

```
    pub description: Option<String>,
    pub poster_url: Option<String>,
    pub release_date: Option<chrono::NaiveDate>,
}

#[derive(Deserialize)]
pub struct SearchParams {
    pub q: Option<String>,
}
```

dengan ket:

- `Movie`: Database model
- `CreateMovieRequest`: untuk Create Request
- `UpdateMovieRequest`: untuk Update Request
- `SearchParams`: Query parameters untuk pencarian film

**Fields:**

- `id`, `title`, `genre`, `rating`, `duration`, `description`, `poster_url`, `release_date`

**src/models/showtime.rs**

Membuat model Showtimes:

```
use serde::{Deserialize, Serialize};
use sqlx::FromRow;

#[derive(Serialize, FromRow, Clone)]
pub struct Showtime {
    pub id: i64,
    pub movie_id: Option<i64>,
    pub studio_id: Option<i64>,
    pub start_time: Option<chrono::NaiveDateTime>,
    pub price: Option<rust_decimal::Decimal>,
}

#[derive(Deserialize)]
pub struct CreateShowtimeRequest {
    pub movie_id: i64,
    pub studio_id: i64,
    pub start_time: chrono::NaiveDateTime,
    pub price: rust_decimal::Decimal,
}

#[derive(Deserialize)]
pub struct UpdateShowtimeRequest {
    pub movie_id: Option<i64>,
    pub studio_id: Option<i64>,
    pub start_time: Option<chrono::NaiveDateTime>,
```

```
026     pub price: Option<rust_decimal::Decimal>,
027 }
```

dengan ket:

- `Showtime`: Database model
- `CreateShowtimeRequest`: untuk Create Request
- `UpdateShowtimeRequest`: untuk Update Request

**Fields:**

- `id`, `movie_id`, `studio_id`, `start_time`, `price`

**src/models/studio.rs**

Membuat model untuk Studios:

```
001 use serde::{Deserialize, Serialize};
002 use sqlx::FromRow;
003
004 #[derive(Serialize, FromRow, Clone)]
005 pub struct Studio {
006     pub id: i64,
007     pub cinema_id: Option<i64>,
008     pub name: String,
009     pub capacity: i32,
010     pub r#type: Option<String>,
011 }
012
013 #[derive(Deserialize)]
014 pub struct CreateStudioRequest {
015     pub cinema_id: i64,
016     pub name: String,
017     pub capacity: i32,
018     pub r#type: Option<String>,
019 }
020
021 #[derive(Deserialize)]
022 pub struct UpdateStudioRequest {
023     pub cinema_id: Option<i64>,
024     pub name: Option<String>,
025     pub capacity: Option<i32>,
026     pub r#type: Option<String>,
027 }
```

dengan ket:

- `Studio`: Database model
- `CreateStudioRequest`: untuk Create Request
- `UpdateStudioRequest`: untuk Update Request

**Fields:**

- `id`, `cinema_id`, `name`, `capacity`, `type`

**src/models/seat.rs**

Membuat model untuk Seats:

```rust
use serde::{Deserialize, Serialize};
use sqlx::FromRow;

#[derive(Serialize, FromRow, Clone)]
pub struct Seat {
    pub id: i64,
    pub studio_id: i64,
    pub seat_code: String,
    pub seat_row: Option<i32>,
    pub seat_col: Option<i32>,
    pub seat_status: Option<String>,
}

#[derive(Serialize, Clone)]
pub struct SeatWithBookingStatus {
    pub id: i64,
    pub studio_id: i64,
    pub seat_code: String,
    pub seat_row: Option<i32>,
    pub seat_col: Option<i32>,
    pub seat_status: Option<String>,
    pub is_booked: bool,
    pub booking_id: Option<i64>,
}

#[derive(Deserialize)]
pub struct GenerateSeatsRequest {
    pub studio_id: i64,
    pub rows: i32,
    pub seats_per_row: i32,
}

#[derive(Serialize)]
pub struct GenerateSeatsResponse {
    pub studio_id: i64,
    pub total_seats_created: i32,
}
```

dengan ket:

- `Seat`: Database model untuk kursi
- `SeatWithBookingStatus`: Extended model dengan status booking
- `GenerateSeatsRequest`: Fungsi untuk auto-generate kursi

**Fields:**

- `id`, `studio_id`, `seat_code`, `seat_row`, `seat_col`, `seat_status`

**src/models/booking.rs**

Membuat model untuk Bookings:

```rust
use serde::{Deserialize, Serialize};
use sqlx::FromRow;

#[derive(Serialize, FromRow, Clone)]
pub struct Booking {
    pub id: i64,
    pub user_id: Option<i64>,
    pub showtime_id: Option<i64>,
    pub booking_code: Option<String>,
    pub total_price: Option<rust_decimal::Decimal>,
    pub payment_status: Option<String>,
    #[sqlx(default)]
    pub created_at: Option<chrono::NaiveDateTime>,
}

#[derive(Serialize, FromRow, Clone)]
pub struct BookingSeat {
    pub id: i64,
    pub booking_id: Option<i64>,
    pub seat_id: Option<i64>,
    pub price: Option<rust_decimal::Decimal>,
}

#[derive(Deserialize)]
pub struct CreateBookingRequest {
    pub user_id: i64,
    pub showtime_id: i64,
    pub seat_ids: Vec<i64>,
}

#[derive(Deserialize)]
pub struct UpdatePaymentStatusRequest {
    pub payment_status: String,
}

#[derive(Serialize)]
pub struct BookingDetail {
    pub id: i64,
    pub user_id: Option<i64>,
    pub showtime_id: Option<i64>,
    pub booking_code: Option<String>,
    pub total_price: Option<rust_decimal::Decimal>,
    pub payment_status: Option<String>,
    pub created_at: Option<chrono::NaiveDateTime>,
```

```
045      pub seats: Vec<BookingSeatDetail>,
046  }
047
048  #[derive(Serialize, Clone)]
049  pub struct BookingSeatDetail {
050      pub seat_id: i64,
051      pub seat_code: String,
052      pub price: Option<rust_decimal::Decimal>,
053  }
```

dengan ket:

- `Booking`: Database model
- `BookingSeat`: Relasi booking dengan seat
- `CreateBookingRequest`: Fungsi booking dengan banyak kursi
- `UpdatePaymentStatusRequest`: Fngsi untuk update payment
- `BookingDetail`: Response detail book
- `BookingSeatDetail`: Detail kursi dalam booking

**Fields:**

- `id`, `user_id`, `showtime_id`, `booking_code`, `total_price`, `payment_status`, `created_at`

---

## Handlers Layer (Business Logic)

Semua handlers diimplementasikan dengan:

- `.map()` untuk transformasi data
- `.unwrap_or_else()` untuk error handling
- Pattern matching dengan `match`
- Immutable transformations

**src/handlers/movie_handler.rs**

CRUD operations untuk Movies:

**Functions:**

- `get_all_movies()`: Fetch semua film
- `search_movies()`: Search dengan query parameter
- `create_movie()`: Insert film baru
- `update_movie()`: Update partial fields
- `delete_movie()`: Delete film

**SC:**

```
001  use axum::{extract::{Path, Query, State}, Json};
002  use sqlx::MySqlPool;
003  use crate::models::*;
```

```
004
005  pub async fn get_all_movies(
006      State(pool): State<MySqlPool>,
007  ) -> Json<ApiResponse<Vec<Movie>>> {
008      sqlx::query_as::<_, Movie>(
009          "SELECT id, title, genre, rating, duration, description, poster_url,
release_date FROM movies"
010      )
011      .fetch_all(&pool)
012      .await
013      .map(|movies| Json(ApiResponse::success("Berhasil mengambil semua film",
movies)))
014      .unwrap_or_else(|e| Json(ApiResponse::error(&format!("Database error:
{}", e))))
015  }
016
017  pub async fn search_movies(
018      State(pool): State<MySqlPool>,
019      Query(params): Query<SearchParams>,
020  ) -> Json<ApiResponse<Vec<Movie>>> {
021      let search_pattern = params.q
022          .map(|query_str| format!("%{}%", query_str))
023          .unwrap_or_else(|| "%".to_string());
024
025      sqlx::query_as::<_, Movie>(
026          "SELECT id, title, genre, rating, duration, description, poster_url,
release_date FROM movies WHERE title LIKE ?"
027      )
028      .bind(search_pattern)
029      .fetch_all(&pool)
030      .await
031      .map(|movies| Json(ApiResponse::success("Berhasil mencari film",
movies)))
032      .unwrap_or_else(|e| Json(ApiResponse::error(&format!("Database error:
{}", e))))
033  }
034
035  pub async fn create_movie(
036      State(pool): State<MySqlPool>,
037      Json(payload): Json<CreateMovieRequest>,
038  ) -> Json<ApiResponse<Movie>> {
039      let insert_result = sqlx::query(
040          "INSERT INTO movies (title, genre, rating, duration, description,
poster_url, release_date) VALUES (?, ?, ?, ?, ?, ?, ?)"
041      )
042      .bind(&payload.title)
043      .bind(&payload.genre)
044      .bind(&payload.rating)
045      .bind(payload.duration)
046      .bind(&payload.description)
047      .bind(&payload.poster_url)
048      .bind(payload.release_date)
049      .execute(&pool)
050      .await;
```

```
051
052      match insert_result {
053          Ok(result) => {
054              let movie_id = result.last_insert_id() as i64;
055
056              sqlx::query_as::<_, Movie>(
057                  "SELECT id, title, genre, rating, duration, description,
poster_url, release_date FROM movies WHERE id = ?"
058              )
059              .bind(movie_id)
060              .fetch_one(&pool)
061              .await
062              .map(|movie| Json(ApiResponse::success("Berhasil menambahkan
film", movie)))
063              .unwrap_or_else(|e| Json(ApiResponse::error(&format!("Failed to
fetch created movie: {}", e))))
064          },
065          Err(e) => Json(ApiResponse::error(&format!("Database error: {}", e)))
066      }
067  }
068
069  pub async fn update_movie(
070      State(pool): State<MySqlPool>,
071      Path(id): Path<i64>,
072      Json(payload): Json<UpdateMovieRequest>,
073  ) -> Json<ApiResponse<Movie>> {
074      let movie_exists = sqlx::query_as::<_, Movie>(
075          "SELECT id, title, genre, rating, duration, description, poster_url,
release_date FROM movies WHERE id = ?"
076      )
077      .bind(id)
078      .fetch_optional(&pool)
079      .await;
080
081      match movie_exists {
082          Ok(Some(existing_movie)) => {
083              let updated_title =
payload.title.unwrap_or(existing_movie.title);
084              let updated_genre = payload.genre.or(existing_movie.genre);
085              let updated_rating = payload.rating.or(existing_movie.rating);
086              let updated_duration =
payload.duration.or(existing_movie.duration);
087              let updated_description =
payload.description.or(existing_movie.description);
088              let updated_poster_url =
payload.poster_url.or(existing_movie.poster_url);
089              let updated_release_date =
payload.release_date.or(existing_movie.release_date);
090
091              sqlx::query(
092                  "UPDATE movies SET title = ?, genre = ?, rating = ?, duration
= ?, description = ?, poster_url = ?, release_date = ? WHERE id = ?"
093              )
094              .bind(&updated_title)
```

```
095             .bind(&updated_genre)
096             .bind(&updated_rating)
097             .bind(updated_duration)
098             .bind(&updated_description)
099             .bind(&updated_poster_url)
100             .bind(updated_release_date)
101             .bind(id)
102             .execute(&pool)
103             .await
104             .ok();
105 
106             sqlx::query_as::<_, Movie>(
107                 "SELECT id, title, genre, rating, duration, description,
poster_url, release_date FROM movies WHERE id = ?"
108             )
109             .bind(id)
110             .fetch_one(&pool)
111             .await
112             .map(|movie| Json(ApiResponse::success("Berhasil mengupdate
film", movie)))
113             .unwrap_or_else(|e| Json(ApiResponse::error(&format!("Failed to
fetch updated movie: {}", e))))
114         },
115         Ok(None) => Json(ApiResponse::error(&format!("Film dengan id {} tidak
ditemukan", id))),
116         Err(e) => Json(ApiResponse::error(&format!("Database error: {}", e)))
117     }
118 }
119 
120 pub async fn delete_movie(
121     State(pool): State<MySqlPool>,
122     Path(id): Path<i64>,
123 ) -> Json<ApiResponse<DeleteResponse>> {
124     let movie_check = sqlx::query_as::<_, Movie>(
125         "SELECT id, title, genre, rating, duration, description, poster_url,
release_date FROM movies WHERE id = ?"
126     )
127     .bind(id)
128     .fetch_optional(&pool)
129     .await;
130 
131     match movie_check {
132         Ok(Some(_)) => {
133             sqlx::query("DELETE FROM movies WHERE id = ?")
134                 .bind(id)
135                 .execute(&pool)
136                 .await
137                 .map(|_| Json(ApiResponse::success(
138                     "Berhasil menghapus film",
139                     DeleteResponse { id, deleted: true }
140                 )))
141                 .unwrap_or_else(|e| Json(ApiResponse::error(&format!("Failed
to delete movie: {}", e))))
142         },
```

```
143          Ok(None) => Json(ApiResponse::error(&format!("Film dengan id {} tidak
ditemukan", id))),
144          Err(e) => Json(ApiResponse::error(&format!("Database error: {}", e)))
145      }
146  }
```

**src/handlers/showtime_handler.rs**

CRUD operations untuk Showtimes:

**Functions:**

- `get_all_showtimes()`: Fetch semua jadwal
- `get_showtimes_by_movie()`: Filter by movie_id
- `create_showtime()`: Insert jadwal baru
- `update_showtime()`: Update jadwal
- `delete_showtime()`: Delete jadwal

**SC:**

```
001  use axum::{extract::{Path, State}, Json};
002  use sqlx::MySqlPool;
003  use crate::models::*;
004
005  pub async fn get_all_showtimes(
006      State(pool): State<MySqlPool>,
007  ) -> Json<ApiResponse<Vec<Showtime>>> {
008      sqlx::query_as::<_, Showtime>(
009          "SELECT id, movie_id, studio_id, start_time, price FROM showtimes"
010      )
011      .fetch_all(&pool)
012      .await
013      .map(|showtimes| Json(ApiResponse::success("Berhasil mengambil semua
showtimes", showtimes)))
014      .unwrap_or_else(|e| Json(ApiResponse::error(&format!("Database error:
{}", e))))
015  }
016
017  pub async fn get_showtimes_by_movie(
018      State(pool): State<MySqlPool>,
019      Path(movie_id): Path<i64>,
020  ) -> Json<ApiResponse<Vec<Showtime>>> {
021      sqlx::query_as::<_, Showtime>(
022          "SELECT id, movie_id, studio_id, start_time, price FROM showtimes
WHERE movie_id = ?"
023      )
024      .bind(movie_id)
025      .fetch_all(&pool)
026      .await
027      .map(|showtimes| Json(ApiResponse::success("Berhasil mengambil showtimes
untuk film ini", showtimes)))
```

```
028      .unwrap_or_else(|e| Json(ApiResponse::error(&format!("Database error:
{}", e))))
029  }
030
031  pub async fn create_showtime(
032      State(pool): State<MySqlPool>,
033      Json(payload): Json<CreateShowtimeRequest>,
034  ) -> Json<ApiResponse<Showtime>> {
035      let insert_result = sqlx::query(
036          "INSERT INTO showtimes (movie_id, studio_id, start_time, price)
VALUES (?, ?, ?, ?)"
037      )
038      .bind(payload.movie_id)
039      .bind(payload.studio_id)
040      .bind(payload.start_time)
041      .bind(payload.price)
042      .execute(&pool)
043      .await;
044
045      match insert_result {
046          Ok(result) => {
047              let showtime_id = result.last_insert_id() as i64;
048
049              sqlx::query_as::<_, Showtime>(
050                  "SELECT id, movie_id, studio_id, start_time, price FROM
showtimes WHERE id = ?"
051              )
052              .bind(showtime_id)
053              .fetch_one(&pool)
054              .await
055              .map(|showtime| Json(ApiResponse::success("Berhasil menambahkan
showtime", showtime)))
056              .unwrap_or_else(|e| Json(ApiResponse::error(&format!("Failed to
fetch created showtime: {}", e))))
057          },
058          Err(e) => Json(ApiResponse::error(&format!("Database error: {}", e)))
059      }
060  }
061
062  pub async fn update_showtime(
063      State(pool): State<MySqlPool>,
064      Path(id): Path<i64>,
065      Json(payload): Json<UpdateShowtimeRequest>,
066  ) -> Json<ApiResponse<Showtime>> {
067      let showtime_exists = sqlx::query_as::<_, Showtime>(
068          "SELECT id, movie_id, studio_id, start_time, price FROM showtimes
WHERE id = ?"
069      )
070      .bind(id)
071      .fetch_optional(&pool)
072      .await;
073
074      match showtime_exists {
075          Ok(Some(existing_showtime)) => {
```

```
076             let updated_movie_id =
payload.movie_id.or(existing_showtime.movie_id);
077             let updated_studio_id =
payload.studio_id.or(existing_showtime.studio_id);
078             let updated_start_time =
payload.start_time.or(existing_showtime.start_time);
079             let updated_price = payload.price.or(existing_showtime.price);
080
081             sqlx::query(
082                 "UPDATE showtimes SET movie_id = ?, studio_id = ?, start_time
= ?, price = ? WHERE id = ?"
083             )
084             .bind(updated_movie_id)
085             .bind(updated_studio_id)
086             .bind(updated_start_time)
087             .bind(updated_price)
088             .bind(id)
089             .execute(&pool)
090             .await
091             .ok();
092
093             sqlx::query_as::<_, Showtime>(
094                 "SELECT id, movie_id, studio_id, start_time, price FROM
showtimes WHERE id = ?"
095             )
096             .bind(id)
097             .fetch_one(&pool)
098             .await
099             .map(|showtime| Json(ApiResponse::success("Berhasil mengupdate
showtime", showtime)))
100             .unwrap_or_else(|e| Json(ApiResponse::error(&format!("Failed to
fetch updated showtime: {}", e))))
101         },
102         Ok(None) => Json(ApiResponse::error(&format!("Showtime dengan id {}
tidak ditemukan", id))),
103         Err(e) => Json(ApiResponse::error(&format!("Database error: {}", e)))
104     }
105 }
106
107 pub async fn delete_showtime(
108     State(pool): State<MySqlPool>,
109     Path(id): Path<i64>,
110 ) -> Json<ApiResponse<DeleteResponse>> {
111     sqlx::query("DELETE FROM showtimes WHERE id = ?")
112         .bind(id)
113         .execute(&pool)
114         .await
115         .map(|result| {
116             let deleted = result.rows_affected() > 0;
117             if deleted {
118                 Json(ApiResponse::success("Berhasil menghapus showtime",
DeleteResponse { id, deleted }))
119             } else {
120                 Json(ApiResponse::error(&format!("Showtime dengan id {} tidak
```

```
ditemukan", id)))
121             }
122         })
123         .unwrap_or_else(|e| Json(ApiResponse::error(&format!("Database error:
{}", e))))
124 }
```

**src/handlers/studio_handler.rs**

CRUD operations untuk Studios:

**Functions:**

- `get_all_studios()`: Fetch semua studio
- `get_studio_by_id()`: Get single studio
- `get_studios_by_cinema()`: Filter by cinema_id
- `create_studio()`: Insert studio baru
- `update_studio()`: Update studio data
- `delete_studio()`: Delete studio

**SC:**

```
001 use axum::{extract::{Path, State}, Json};
002 use sqlx::MySqlPool;
003 use crate::models::*;
004
005 // Get all studios
006 pub async fn get_all_studios(
007     State(pool): State<MySqlPool>,
008 ) -> Json<ApiResponse<Vec<Studio>>> {
009     sqlx::query_as::<_, Studio>(
010         "SELECT id, cinema_id, name, capacity, type FROM studios"
011     )
012     .fetch_all(&pool)
013     .await
014     .map(|studios| Json(ApiResponse::success("Berhasil mengambil semua
studio", studios)))
015     .unwrap_or_else(|e| Json(ApiResponse::error(&format!("Database error:
{}", e))))
016 }
017
018 // Get studio by ID
019 pub async fn get_studio_by_id(
020     State(pool): State<MySqlPool>,
021     Path(id): Path<i64>,
022 ) -> Json<ApiResponse<Studio>> {
023     sqlx::query_as::<_, Studio>(
024         "SELECT id, cinema_id, name, capacity, type FROM studios WHERE id =
?"
025     )
026     .bind(id)
```

```
027         .fetch_one(&pool)
028         .await
029         .map(|studio| Json(ApiResponse::success("Berhasil mengambil studio",
    studio)))
030         .unwrap_or_else(|e| Json(ApiResponse::error(&format!("Studio tidak
    ditemukan: {}", e))))
031     }
032
033     // Get studios by cinema_id
034     pub async fn get_studios_by_cinema(
035         State(pool): State<MySqlPool>,
036         Path(cinema_id): Path<i64>,
037     ) -> Json<ApiResponse<Vec<Studio>>> {
038         sqlx::query_as::<_, Studio>(
039             "SELECT id, cinema_id, name, capacity, type FROM studios WHERE
    cinema_id = ?"
040         )
041         .bind(cinema_id)
042         .fetch_all(&pool)
043         .await
044         .map(|studios| Json(ApiResponse::success("Berhasil mengambil studio untuk
    cinema ini", studios)))
045         .unwrap_or_else(|e| Json(ApiResponse::error(&format!("Database error:
    {}", e))))
046     }
047
048     // Create studio
049     pub async fn create_studio(
050         State(pool): State<MySqlPool>,
051         Json(payload): Json<CreateStudioRequest>,
052     ) -> Json<ApiResponse<Studio>> {
053         let insert_result = sqlx::query(
054             "INSERT INTO studios (cinema_id, name, capacity, type) VALUES (?, ?,
    ?, ?)"
055         )
056         .bind(payload.cinema_id)
057         .bind(&payload.name)
058         .bind(payload.capacity)
059         .bind(&payload.r#type)
060         .execute(&pool)
061         .await;
062
063         match insert_result {
064             Ok(result) => {
065                 let studio_id = result.last_insert_id() as i64;
066
067                 sqlx::query_as::<_, Studio>(
068                     "SELECT id, cinema_id, name, capacity, type FROM studios
    WHERE id = ?"
069                 )
070                 .bind(studio_id)
071                 .fetch_one(&pool)
072                 .await
073                 .map(|studio| Json(ApiResponse::success("Berhasil menambahkan
```

```
studio", studio)))
074             .unwrap_or_else(|e| Json(ApiResponse::error(&format!("Failed to
fetch created studio: {}", e))))
075         },
076         Err(e) => Json(ApiResponse::error(&format!("Database error: {}", e)))
077     }
078 }
079
080 // Update studio
081 pub async fn update_studio(
082     State(pool): State<MySqlPool>,
083     Path(id): Path<i64>,
084     Json(payload): Json<UpdateStudioRequest>,
085 ) -> Json<ApiResponse<Studio>> {
086     let studio_exists = sqlx::query_as::<_, Studio>(
087         "SELECT id, cinema_id, name, capacity, type FROM studios WHERE id =
?"
088     )
089     .bind(id)
090     .fetch_optional(&pool)
091     .await;
092
093     match studio_exists {
094         Ok(Some(existing_studio)) => {
095             let updated_cinema_id =
payload.cinema_id.or(existing_studio.cinema_id);
096             let updated_name = payload.name.unwrap_or(existing_studio.name);
097             let updated_capacity =
payload.capacity.unwrap_or(existing_studio.capacity);
098             let updated_type = payload.r#type.or(existing_studio.r#type);
099
100             sqlx::query(
101                 "UPDATE studios SET cinema_id = ?, name = ?, capacity = ?,
type = ? WHERE id = ?"
102             )
103             .bind(updated_cinema_id)
104             .bind(&updated_name)
105             .bind(updated_capacity)
106             .bind(&updated_type)
107             .bind(id)
108             .execute(&pool)
109             .await
110             .ok();
111
112             sqlx::query_as::<_, Studio>(
113                 "SELECT id, cinema_id, name, capacity, type FROM studios
WHERE id = ?"
114             )
115             .bind(id)
116             .fetch_one(&pool)
117             .await
118             .map(|studio| Json(ApiResponse::success("Berhasil mengupdate
studio", studio)))
119             .unwrap_or_else(|e| Json(ApiResponse::error(&format!("Failed to
```

```
fetch updated studio: {}", e))))
        },
        Ok(None) => Json(ApiResponse::error(&format!("Studio dengan id {}
tidak ditemukan", id))),
        Err(e) => Json(ApiResponse::error(&format!("Database error: {}", e)))
    }
}

// Delete studio
pub async fn delete_studio(
    State(pool): State<MySqlPool>,
    Path(id): Path<i64>,
) -> Json<ApiResponse<DeleteResponse>> {
    sqlx::query("DELETE FROM studios WHERE id = ?")
        .bind(id)
        .execute(&pool)
        .await
        .map(|result| {
            let deleted = result.rows_affected() > 0;
            if deleted {
                Json(ApiResponse::success("Berhasil menghapus studio",
DeleteResponse { id, deleted }))
            } else {
                Json(ApiResponse::error(&format!("Studio dengan id {} tidak
ditemukan", id)))
            }
        })
        .unwrap_or_else(|e| Json(ApiResponse::error(&format!("Database error:
{}", e))))
}
```

**src/handlers/seat_handler.rs**

Operations untuk Seats dengan generator:

**Functions:**

- `get_seats_by_studio()`: Fetch kursi per studio
- `get_seats_by_showtime()`: Fetch kursi untuk showtime tertentu
- `get_available_seats()`: Filter hanya kursi available
- `generate_seats_for_studio()`: **Auto-generate** kursi (A1-A10, B1-B10, dst)

**SC:**

```
use axum::{extract::{Path, State}, Json};
use sqlx::MySqlPool;
use crate::models::*;

// Get all seats
pub async fn get_all_seats(
    State(pool): State<MySqlPool>,
```

```
008  ) -> Json<ApiResponse<Vec<Seat>>> {
009      sqlx::query_as::<_, Seat>(
010          "SELECT id, studio_id, seat_code, seat_row, seat_col, seat_status
FROM seats"
011      )
012      .fetch_all(&pool)
013      .await
014      .map(|seats| Json(ApiResponse::success("Berhasil mengambil semua seats",
seats)))
015      .unwrap_or_else(|e| Json(ApiResponse::error(&format!("Database error:
{}", e))))
016  }
017
018  // Get seats by studio_id
019  pub async fn get_seats_by_studio(
020      State(pool): State<MySqlPool>,
021      Path(studio_id): Path<i64>,
022  ) -> Json<ApiResponse<Vec<Seat>>> {
023      sqlx::query_as::<_, Seat>(
024          "SELECT id, studio_id, seat_code, seat_row, seat_col, seat_status
FROM seats WHERE studio_id = ? ORDER BY seat_row, seat_col"
025      )
026      .bind(studio_id)
027      .fetch_all(&pool)
028      .await
029      .map(|seats| Json(ApiResponse::success("Berhasil mengambil seats untuk
studio ini", seats)))
030      .unwrap_or_else(|e| Json(ApiResponse::error(&format!("Database error:
{}", e))))
031  }
032
033  // Get seats by showtime
034  pub async fn get_seats_by_showtime(
035      State(pool): State<MySqlPool>,
036      Path(showtime_id): Path<i64>,
037  ) -> Json<ApiResponse<Vec<SeatWithBookingStatus>>> {
039      sqlx::query_as::<_, (i64, i64, String, Option<i32>, Option<i32>,
Option<String>, Option<i64>)>(
040          "SELECT
041              s.id,
042              s.studio_id,
043              s.seat_code,
044              s.seat_row,
045              s.seat_col,
046              s.seat_status,
047              bs.booking_id
048          FROM seats s
049          JOIN showtimes st ON s.studio_id = st.studio_id
050          LEFT JOIN booking_seats bs ON bs.seat_id = s.id
051              AND bs.booking_id IN (
052                  SELECT b.id FROM bookings b WHERE b.showtime_id = st.id
053              )
054          WHERE st.id = ?
055          ORDER BY s.seat_row, s.seat_col"
```

```
    )
    .bind(showtime_id)
    .fetch_all(&pool)
    .await
    .map(|rows| {

        rows.into_iter()
            .map(|(id, studio_id, seat_code, seat_row, seat_col, seat_status,
booking_id)| {
                SeatWithBookingStatus {
                    id,
                    studio_id,
                    seat_code,
                    seat_row,
                    seat_col,
                    seat_status,
                    is_booked: booking_id.is_some(),
                    booking_id,
                }
            })
            .collect::<Vec<_>>()
    })
    .map(|seats| Json(ApiResponse::success("Berhasil mengambil seats dengan
status booking", seats)))
    .unwrap_or_else(|e| Json(ApiResponse::error(&format!("Database error:
{}", e))))
}

// Get available seats showtime
pub async fn get_available_seats_by_showtime(
    State(pool): State<MySqlPool>,
    Path(showtime_id): Path<i64>,
) -> Json<ApiResponse<Vec<Seat>>> {
    sqlx::query_as::<_, Seat>(
        "SELECT DISTINCT
            s.id,
            s.studio_id,
            s.seat_code,
            s.seat_row,
            s.seat_col,
            s.seat_status
        FROM seats s
        JOIN showtimes st ON s.studio_id = st.studio_id
        LEFT JOIN booking_seats bs ON bs.seat_id = s.id
            AND bs.booking_id IN (
                SELECT b.id FROM bookings b WHERE b.showtime_id = st.id
            )
        WHERE st.id = ?
            AND bs.booking_id IS NULL
            AND s.seat_status = 'AVAILABLE'
        ORDER BY s.seat_row, s.seat_col"
    )
    .bind(showtime_id)
    .fetch_all(&pool)
```

```
107      .await
108      .map(|seats| Json(ApiResponse::success("Berhasil mengambil seats yang
tersedia", seats)))
109      .unwrap_or_else(|e| Json(ApiResponse::error(&format!("Database error:
{}", e))))
110  }
111
112  // Generate seats studio
113  pub async fn generate_seats_for_studio(
114      State(pool): State<MySqlPool>,
115      Json(payload): Json<GenerateSeatsRequest>,
116  ) -> Json<ApiResponse<GenerateSeatsResponse>> {
117      let studio_check = sqlx::query_scalar::<_, i64>(
118          "SELECT COUNT(*) FROM studios WHERE id = ?"
119      )
120      .bind(payload.studio_id)
121      .fetch_one(&pool)
122      .await;
123
124      match studio_check {
125          Ok(count) if count > 0 => {
126              let seat_codes: Vec<(String, i32, i32)> = (1..=payload.rows)
127                  .flat_map(|row| {
128                      let row_letter = char::from((b'A' + (row - 1) as
u8)).to_string();
129                      (1..=payload.seats_per_row)
130                          .map(move |col| {
131                              (format!("{}{}", row_letter, col), row, col)
132                          })
133                          .collect::<Vec<_>>()
134                  })
135                  .collect();
136
137              let mut total_inserted = 0;
138              for (seat_code, row, col) in seat_codes {
139                  let result = sqlx::query(
140                      "INSERT INTO seats (studio_id, seat_code, seat_row,
seat_col, seat_status) VALUES (?, ?, ?, ?, 'AVAILABLE')"
141                  )
142                  .bind(payload.studio_id)
143                  .bind(&seat_code)
144                  .bind(row)
145                  .bind(col)
146                  .execute(&pool)
147                  .await;
148
149                  if result.is_ok() {
150                      total_inserted += 1;
151                  }
152              }
153
154              Json(ApiResponse::success(
155                  &format!("Berhasil generate {} kursi untuk studio {}",
total_inserted, payload.studio_id),
```

```
                GenerateSeatsResponse {
                    studio_id: payload.studio_id,
                    total_seats_created: total_inserted,
                }
            ))
        },
        Ok(_) => Json(ApiResponse::error(&format!("Studio dengan id {} tidak ditemukan", payload.studio_id))),
        Err(e) => Json(ApiResponse::error(&format!("Database error: {}", e)))
    }
}
```

**src/handlers/booking_handler.rs**

Operations untuk Booking film:

**Functions:**

- `get_all_bookings()`: Fetch semua booking dengan detail seats
- `get_booking_by_id()`: Get single booking + seats (JOIN query)
- `get_bookings_by_user()`: Filter by user_id
- `create_booking()`: **Multi-seat booking** dengan validasi
- `update_payment_status()`: Update PENDING → PAID → CANCELLED
- `cancel_booking()`: Cancel booking + kembalikan seat status

**SC:**

```
use axum::{extract::{Path, State}, Json};
use sqlx::MySqlPool;
use crate::models::*;
use std::time::{SystemTime, UNIX_EPOCH};

// Generate unique booking code
fn generate_booking_code() -> String {
    let timestamp = SystemTime::now()
        .duration_since(UNIX_EPOCH)
        .unwrap()
        .as_secs();
    format!("BK{}", timestamp)
}

// Get all bookings
pub async fn get_all_bookings(
    State(pool): State<MySqlPool>,
) -> Json<ApiResponse<Vec<Booking>>> {
    sqlx::query_as::<_, Booking>(
        "SELECT id, user_id, showtime_id, booking_code, total_price, payment_status, CAST(created_at AS DATETIME) as created_at FROM bookings ORDER BY created_at DESC"
    )
    .fetch_all(&pool)
```

```
023         .await
024         .map(|bookings| Json(ApiResponse::success("Berhasil mengambil semua
bookings", bookings)))
025         .unwrap_or_else(|e| Json(ApiResponse::error(&format!("Database error:
{}", e))))
026     }
027
028     // Get booking by ID
029     pub async fn get_booking_by_id(
030         State(pool): State<MySqlPool>,
031         Path(id): Path<i64>,
032     ) -> Json<ApiResponse<BookingDetail>> {
033         let booking_result = sqlx::query_as::<_, Booking>(
034             "SELECT id, user_id, showtime_id, booking_code, total_price,
payment_status, CAST(created_at AS DATETIME) as created_at FROM bookings WHERE id
= ?"
035         )
036         .bind(id)
037         .fetch_optional(&pool)
038         .await;
039
040         match booking_result {
041             Ok(Some(booking)) => {
042                 let seats_result = sqlx::query_as::<_, (i64, String,
Option<rust_decimal::Decimal>)>(
043                     "SELECT bs.seat_id, s.seat_code, bs.price
044                     FROM booking_seats bs
045                     JOIN seats s ON bs.seat_id = s.id
046                     WHERE bs.booking_id = ?"
047                 )
048                 .bind(id)
049                 .fetch_all(&pool)
050                 .await
051                 .map(|rows| {
052                     rows.into_iter()
053                         .map(|(seat_id, seat_code, price)| BookingSeatDetail {
054                             seat_id,
055                             seat_code,
056                             price,
057                         })
058                         .collect::<Vec<_>>()
059                 });
060
061                 match seats_result {
062                     Ok(seats) => {
063                         let detail = BookingDetail {
064                             id: booking.id,
065                             user_id: booking.user_id,
066                             showtime_id: booking.showtime_id,
067                             booking_code: booking.booking_code,
068                             total_price: booking.total_price,
069                             payment_status: booking.payment_status,
070                             created_at: booking.created_at,
071                             seats,
```

```
                    };
                    Json(ApiResponse::success("Berhasil mengambil detail booking", detail))
                },
                Err(e) => Json(ApiResponse::error(&format!("Failed to fetch seats: {}", e)))
            }
        },
        Ok(None) => Json(ApiResponse::error(&format!("Booking dengan id {} tidak ditemukan", id))),
        Err(e) => Json(ApiResponse::error(&format!("Database error: {}", e)))
    }
}

// Get bookings by user_id
pub async fn get_bookings_by_user(
    State(pool): State<MySqlPool>,
    Path(user_id): Path<i64>,
) -> Json<ApiResponse<Vec<Booking>>> {
    sqlx::query_as::<_, Booking>(
        "SELECT id, user_id, showtime_id, booking_code, total_price, payment_status, CAST(created_at AS DATETIME) as created_at FROM bookings WHERE user_id = ? ORDER BY created_at DESC"
    )
    .bind(user_id)
    .fetch_all(&pool)
    .await
    .map(|bookings| Json(ApiResponse::success("Berhasil mengambil bookings user", bookings)))
    .unwrap_or_else(|e| Json(ApiResponse::error(&format!("Database error: {}", e))))
}

// Create booking
pub async fn create_booking(
    State(pool): State<MySqlPool>,
    Json(payload): Json<CreateBookingRequest>,
) -> Json<ApiResponse<BookingDetail>> {
    let seat_ids_str = payload.seat_ids.iter()
        .map(|id| id.to_string())
        .collect::<Vec<_>>()
        .join(",");

    let booked_seats_check = sqlx::query_scalar::<_, i64>(
        &format!(
            "SELECT COUNT(*) FROM booking_seats bs
            JOIN bookings b ON bs.booking_id = b.id
            WHERE bs.seat_id IN ({})
            AND b.showtime_id = ?
            AND b.payment_status != 'CANCELLED'",
            seat_ids_str
        )
    )
    .bind(payload.showtime_id)
```

```
119         .fetch_one(&pool)
120         .await;
121 
122         match booked_seats_check {
123             Ok(count) if count > 0 => {
124                 return Json(ApiResponse::error("Beberapa kursi sudah
dibooking"));
125             },
126             Err(e) => {
127                 return Json(ApiResponse::error(&format!("Error checking seats:
{}", e)));
128             },
129             _ => {}
130         }
131 
132         let price_result = sqlx::query_scalar::<_, rust_decimal::Decimal>(
133             "SELECT price FROM showtimes WHERE id = ?"
134         )
135         .bind(payload.showtime_id)
136         .fetch_optional(&pool)
137         .await;
138 
139         match price_result {
140             Ok(Some(price)) => {
141                 let total_price = price *
rust_decimal::Decimal::from(payload.seat_ids.len() as i32);
142                 let booking_code = generate_booking_code();
143 
144                 let insert_result = sqlx::query(
145                     "INSERT INTO bookings (user_id, showtime_id, booking_code,
total_price, payment_status) VALUES (?, ?, ?, ?, 'PENDING')"
146                 )
147                 .bind(payload.user_id)
148                 .bind(payload.showtime_id)
149                 .bind(&booking_code)
150                 .bind(total_price)
151                 .execute(&pool)
152                 .await;
153 
154                 match insert_result {
155                     Ok(result) => {
156                         let booking_id = result.last_insert_id() as i64;
157 
158                         let seats_insert_futures = payload.seat_ids.iter()
159                             .map(|seat_id| {
160                                 sqlx::query(
161                                     "INSERT INTO booking_seats (booking_id,
seat_id, price) VALUES (?, ?, ?)"
162                                 )
163                                 .bind(booking_id)
164                                 .bind(seat_id)
165                                 .bind(price)
166                                 .execute(&pool)
167                             });
```

```
168
169                     for future in seats_insert_futures {
170                         future.await.ok();
171                     }
172
173                     for seat_id in payload.seat_ids.iter() {
174                         sqlx::query(
175                             "UPDATE seats SET seat_status = 'booked' WHERE id
= ?"
176                         )
177                         .bind(seat_id)
178                         .execute(&pool)
179                         .await
180                         .ok();
181                     }
182
183                     let booking_detail_result = sqlx::query_as::<_, Booking>(
184                         "SELECT id, user_id, showtime_id, booking_code,
total_price, payment_status, CAST(created_at AS DATETIME) as created_at FROM
bookings WHERE id = ?"
185                     )
186                     .bind(booking_id)
187                     .fetch_one(&pool)
188                     .await;
189
190                     match booking_detail_result {
191                         Ok(booking) => {
192                             let seats_result = sqlx::query_as::<_, (i64,
String, Option<rust_decimal::Decimal>)>(
193                                 "SELECT bs.seat_id, s.seat_code, bs.price
194                                 FROM booking_seats bs
195                                 JOIN seats s ON bs.seat_id = s.id
196                                 WHERE bs.booking_id = ?"
197                             )
198                             .bind(booking_id)
199                             .fetch_all(&pool)
200                             .await
201                             .map(|rows| {
202                                 rows.into_iter()
203                                     .map(|(seat_id, seat_code, price)|
BookingSeatDetail {
204                                         seat_id,
205                                         seat_code,
206                                         price,
207                                     })
208                                     .collect::<Vec<_>>()
209                             })
210                             .unwrap_or_default();
211
212                             let detail = BookingDetail {
213                                 id: booking.id,
214                                 user_id: booking.user_id,
215                                 showtime_id: booking.showtime_id,
216                                 booking_code: booking.booking_code,
```

```
                                    total_price: booking.total_price,
                                    payment_status: booking.payment_status,
                                    created_at: booking.created_at,
                                    seats: seats_result,
                                };

                                Json(ApiResponse::success("Berhasil membuat booking", detail))
                            },
                            Err(e) => Json(ApiResponse::error(&format!("Failed to fetch created booking: {}", e)))
                        }
                    },
                    Err(e) => Json(ApiResponse::error(&format!("Database error: {}", e)))
                }
            },
            Ok(None) => Json(ApiResponse::error(&format!("Showtime dengan id {} tidak ditemukan", payload.showtime_id))),
            Err(e) => Json(ApiResponse::error(&format!("Database error: {}", e)))
        }
    }

    // Update payment status
    pub async fn update_payment_status(
        State(pool): State<MySqlPool>,
        Path(id): Path<i64>,
        Json(payload): Json<UpdatePaymentStatusRequest>,
    ) -> Json<ApiResponse<Booking>> {
        let valid_statuses = vec!["PENDING", "PAID", "CANCELLED"];
        if !valid_statuses.contains(&payload.payment_status.as_str()) {
            return Json(ApiResponse::error("Status payment tidak valid. Harus PENDING, PAID, atau CANCELLED"));
        }

        let update_result = sqlx::query(
            "UPDATE bookings SET payment_status = ? WHERE id = ?"
        )
        .bind(&payload.payment_status)
        .bind(id)
        .execute(&pool)
        .await;

        match update_result {
            Ok(result) if result.rows_affected() > 0 => {
                sqlx::query_as::<_, Booking>(
                    "SELECT id, user_id, showtime_id, booking_code, total_price, payment_status, CAST(created_at AS DATETIME) as created_at FROM bookings WHERE id = ?"
                )
                .bind(id)
                .fetch_one(&pool)
                .await
                .map(|booking| Json(ApiResponse::success("Berhasil update status
```

```
payment", booking)))
264             .unwrap_or_else(|e| Json(ApiResponse::error(&format!("Failed to
fetch updated booking: {}", e))))
265         },
266         Ok(_) => Json(ApiResponse::error(&format!("Booking dengan id {} tidak
ditemukan", id))),
267         Err(e) => Json(ApiResponse::error(&format!("Database error: {}", e)))
268     }
269 }
270
271 // Cancel booking
272 pub async fn cancel_booking(
273     State(pool): State<MySqlPool>,
274     Path(id): Path<i64>,
275 ) -> Json<ApiResponse<Booking>> {
276     let seats_result = sqlx::query_scalar::<_, i64>(
277         "SELECT seat_id FROM booking_seats WHERE booking_id = ?"
278     )
279     .bind(id)
280     .fetch_all(&pool)
281     .await;
282
283
284     let update_result = sqlx::query(
285         "UPDATE bookings SET payment_status = 'CANCELLED' WHERE id = ?"
286     )
287     .bind(id)
288     .execute(&pool)
289     .await;
290
291     match update_result {
292         Ok(result) if result.rows_affected() > 0 => {
293             if let Ok(seat_ids) = seats_result {
294                 for seat_id in seat_ids.iter() {
295                     sqlx::query(
296                         "UPDATE seats SET seat_status = 'available' WHERE id
= ?"
297                     )
298                     .bind(seat_id)
299                     .execute(&pool)
300                     .await
301                     .ok();
302                 }
303             }
304
305             sqlx::query_as::<_, Booking>(
306                 "SELECT id, user_id, showtime_id, booking_code, total_price,
payment_status, CAST(created_at AS DATETIME) as created_at FROM bookings WHERE id
= ?"
307             )
308             .bind(id)
309             .fetch_one(&pool)
310             .await
311             .map(|booking| Json(ApiResponse::success("Berhasil cancel
```

```
booking", booking)))
312             .unwrap_or_else(|e| Json(ApiResponse::error(&format!("Failed to
fetch cancelled booking: {}", e))))
313         },
314         Ok(_) => Json(ApiResponse::error(&format!("Booking dengan id {} tidak
ditemukan", id))),
315         Err(e) => Json(ApiResponse::error(&format!("Database error: {}", e)))
316     }
317 }
```

---

## Routes Layer (Endpoint)

**src/routes/movie_routes.rs**

Membuat endpoint movie dengan method get, post, put dan delete.

```
001 use axum::{routing::{get, post, put, delete}, Router};
002 use sqlx::MySqlPool;
003 use crate::handlers::movie_handler::*;
004
005 pub fn movie_routes() -> Router<MySqlPool> {
006     Router::new()
007         .route("/api/movies/all", get(get_all_movies))
008         .route("/api/movies", get(search_movies).post(create_movie))
009         .route("/api/movies/{id}", put(update_movie).delete(delete_movie))
010 }
```

**src/routes/showtime_routes.rs**

Membuat endpoint showtime dengan method get, post, put dan delete.

```
001 use axum::{routing::{get, post, put, delete}, Router};
002 use sqlx::MySqlPool;
003 use crate::handlers::showtime_handler::*;
004
005 pub fn showtime_routes() -> Router<MySqlPool> {
006     Router::new()
007         .route("/api/showtimes",
get(get_all_showtimes).post(create_showtime))
008         .route("/api/showtimes/movie/{movie_id}",
get(get_showtimes_by_movie))
009         .route("/api/showtimes/{id}",
put(update_showtime).delete(delete_showtime))
010 }
```

**src/routes/studio_routes.rs**

Membuat endpoint studio dengan method get, post, put dan delete.

```
use axum::{routing::{get, post, put, delete}, Router};
use sqlx::MySqlPool;
use crate::handlers::studio_handler::*;

pub fn studio_routes() -> Router<MySqlPool> {
    Router::new()
        .route("/api/studios", get(get_all_studios).post(create_studio))
        .route("/api/studios/{id}",
get(get_studio_by_id).put(update_studio).delete(delete_studio))
        .route("/api/studios/cinema/{cinema_id}", get(get_studios_by_cinema))
}
```

**src/routes/seat_routes.rs**

```
use axum::{routing::{get, post}, Router};
use sqlx::MySqlPool;
use crate::handlers::seat_handler::*;

pub fn seat_routes() -> Router<MySqlPool> {
    Router::new()
        .route("/api/seats", get(get_all_seats))
        .route("/api/seats/generate", post(generate_seats_for_studio))
        .route("/api/seats/studio/{studio_id}", get(get_seats_by_studio))
        .route("/api/seats/showtime/{showtime_id}",
get(get_seats_by_showtime))
        .route("/api/seats/showtime/{showtime_id}/available",
get(get_available_seats_by_showtime))
}
```

**src/routes/booking_routes.rs**

```
use axum::{routing::{get, post, put}, Router};
use sqlx::MySqlPool;
use crate::handlers::booking_handler::*;

pub fn booking_routes() -> Router<MySqlPool> {
    Router::new()
        .route("/api/bookings", get(get_all_bookings).post(create_booking))
        .route("/api/bookings/{id}", get(get_booking_by_id))
        .route("/api/bookings/user/{user_id}", get(get_bookings_by_user))
        .route("/api/bookings/{id}/payment", put(update_payment_status))
        .route("/api/bookings/{id}/cancel", put(cancel_booking))
}
```

---

## Penjelasan Struktur Frontend

**Frontend Architecture (Component-Based)**

Frontend menggunakan Vue.js 3 dengan Composition API dan TailwindCSS untuk styling:

1. **Entry Point (`main.js`)**: Bootstrap Vue application
2. **Root Component (`App.vue`)**: Main layout dan component composition
3. **Components (`components/`)**: Reusable Vue components
4. **Styles (`style.css`)**: Global CSS dan Tailwind directives
5. **Build Tool (`vite.config.js`)**: Vite bundler configuration

---

## File Utama Frontend

**src/main.js**

Entry point yang menginisialisasi Vue application:

```
import { createApp } from "vue";
import "./style.css";
import App from "./App.vue";

createApp(App).mount("#app");
```

**Fungsi:**

- Import Vue 3 core
- Load global styles (termasuk Tailwind)
- Mount App component ke DOM element `#app`

---

**src/App.vue**

Root component yang meng-compose semua sections:

```
<script setup>
import Navbar from "./components/Navbar.vue";
import HeroSection from "./components/HeroSection.vue";
import SearchSection from "./components/SearchSection.vue";
import NowShowingSection from "./components/NowShowingSection.vue";
</script>

<template>
  <div class="min-h-screen bg-gray-50">
    <Navbar />
```

```
      <HeroSection />
      <SearchSection />
      <NowShowingSection />
    </div>
  </template>
```

**Struktur:**

- Composition API dengan `<script setup>`
- Component imports
- Layout dengan Tailwind utility classes

---

**src/components/Navbar.vue**

Navigation bar component:

**Features:**

- Logo Tioskop
- Navigation links (Home, Movies, About)
- Responsive design dengan Tailwind

---

**src/components/HeroSection.vue**

Hero banner section:

**Features:**

- Eye-catching headline
- Call-to-action button
- Background gradient
- Cinema-themed imagery

---

**src/components/SearchSection.vue**

Film search functionality:

**Features:**

- Search input field
- Filter options (genre, date)
- Live search functionality (future: connect to API)
- Responsive grid layout

---

**src/components/NowShowingSection.vue**

Display now showing films:

**Features:**

- Film cards grid
- Fetch data dari `/api/movies/all` endpoint
- Display: poster, title, genre, rating
- Book button untuk setiap film
- Responsive grid (1-2-3-4 columns)

**API Integration:**

```
const fetchMovies = async () => {
  try {
    const response = await fetch("http://127.0.0.1:3000/api/movies/all");
    const data = await response.json();
    if (data.success) {
      movies.value = data.data;
    }
  } catch (error) {
    console.error("Failed to fetch movies:", error);
  }
};
```

---

**vite.config.js**

Vite bundler configuration:

```
import { defineConfig } from "vite";
import vue from "@vitejs/plugin-vue";

export default defineConfig({
  plugins: [vue()],
  server: {
    port: 5173,
    proxy: {
      "/api": {
        target: "http://127.0.0.1:3000",
        changeOrigin: true,
      },
    },
  },
});
```

**Features:**

- Vue plugin integration
- Development server port: 5173
- API proxy untuk backend requests

- Hot Module Replacement (HMR)

---

**package.json**

npm dependencies dan scripts:

```
{
  "name": "tioskop-frontend",
  "version": "1.0.0",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "preview": "vite preview"
  },
  "dependencies": {
    "vue": "^3.4.0"
  },
  "devDependencies": {
    "@vitejs/plugin-vue": "^5.0.0",
    "autoprefixer": "^10.4.0",
    "postcss": "^8.4.0",
    "tailwindcss": "^3.4.0",
    "vite": "^5.0.0"
  }
}
```

**Key Dependencies:**

- **Vue 3**: Progressive JavaScript framework
- **Vite**: Fast build tool dengan HMR
- **TailwindCSS**: Utility-first CSS framework
- **PostCSS**: CSS transformation tool
- **Autoprefixer**: Vendor prefix otomatis

---

## Frontend Development Workflow

### 1. Install Dependencies

```
cd frontend
npm install
```

### 2. Run Development Server

```
npm run dev
```

Server akan berjalan di `http://localhost:5173`

**3. Build for Production**

```
npm run build
```

Generates optimized production build di `dist/` folder

**4. Preview Production Build**

```
npm run preview
```

---

Integration Backend ↔ Frontend

**Data Flow:**

```
Frontend (Vue)  →  HTTP Request  →  Backend (Rust/Axum)
                                          ↓
                                    MySQL Database
                                          ↓
Backend (Rust)  →  JSON Response  →  Frontend (Vue)
                                          ↓
                                    Render UI
```

**API Endpoints Integration:**

| Frontend Action | API Endpoint | Method | Handler |
|---|---|---|---|
| Load movies | `/api/movies/all` | GET | `get_all_movies()` |
| Search movies | `/api/movies?q={query}` | GET | `search_movies()` |
| View showtimes | `/api/showtimes/movie/{id}` | GET | `get_showtimes_by_movie()` |
| Select seats | `/api/seats/showtime/{id}/available` | GET | `get_available_seats()` |
| Create booking | `/api/bookings` | POST | `create_booking()` |

---

# Screenshot

OTW

| Tampilan | Status |
| --- | --- |
| API Get Movies | OTW |
| Daftar Studio + Kursi | OTW |
| Halaman Booking | OTW |
| Response JSON Book Sukses | OTW |

---

# Conclusion

Projek ini menunjukkan bahwa Rust dapat digunakan secara efektif untuk membangun layanan booking bioskop yang memilki kebutuhan:

- Cepat & aman pada sistem concurrency yang tinggi
- Menerapkan paradigma *Functional Programming* dengan sesuai
- Memiliki integritas data kuat melalui sistem booking atomic

Ke depannya, fitur projek ini dapat dikembangkan dengan menambah:

- Payment gateway,
- Notifikasi tiket digital.

---